

ECE 161C: Introduction to Image Processing

Directional Image Analysis and Processing

Rahul Parhi
Assistant Professor, ECE, UCSD

Spring 2026

Outline

- Mathematical Foundations
 - Rotations and the Fourier Transform
 - Radon Transform
 - Rotation of Polynomials
 - Directional Derivatives
- Local Directional Analysis
 - Structure Tensor
- Steerable Filters
 - Derivative-Based Filters
- Edge Detector: Revisited

Directionality in Image Processing

- Importance of directional cues
 - Edges, ridges, patterns, texture
 - Visual perception is orientation-sensitive
 - Neurons in the primary visual cortex have orientation selectivity

(Hubel and Wiesel, 1958)
- Invariant Processing and Feature Detection
 - Invariant operators: Gradient magnitude, Laplacian, ...
- Computational challenges
 - Selectivity to orientation
 - Steerability (orientation can be arbitrary)
 - Separable filters are not orientation-sensitive

Mathematical Foundations

- Rotations and the Fourier Transform
- Radon Transform
- Rotation of Polynomials
- Directional Derivatives

Rotations and the Fourier Transform

Recall: Continuous-domain Fourier transform $\hat{f}(\boldsymbol{\omega}) = \int_{\mathbb{R}^2} f(\boldsymbol{x}) e^{-j\boldsymbol{\omega}^\top \boldsymbol{x}} d\boldsymbol{x}$

Spatial-domain rotations correspond to what in the Fourier domain?

$$\mathbf{R}_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad \begin{array}{c} \text{counterclockwise} \\ \text{rotation} \end{array}$$

$$\begin{array}{ccc} f(\boldsymbol{x}) & \xleftrightarrow{\mathcal{F}} & \hat{f}(\boldsymbol{\omega}) \\ f(\mathbf{R}_\theta \boldsymbol{x}) & \xleftrightarrow{\mathcal{F}} & \hat{f}(\mathbf{R}_\theta \boldsymbol{\omega}) \quad (\text{proof by change-of-variables}) \end{array}$$

Spatial-domain rotations correspond to Fourier-domain rotations

Radon Transform

The Radon transform of $f(x, y)$ corresponds to all line integrals of $f(x, y)$

Notation: $\boldsymbol{\theta} = (\cos \theta, \sin \theta) \in \mathbb{R}^2$

A line in $\mathbb{R}^2$ can be represented by all $\boldsymbol{x} \in \mathbb{R}^2$ such that

$$\boldsymbol{\theta}^\top \boldsymbol{x} = t \quad \Leftrightarrow \quad x \cos \theta + y \sin \theta = t$$

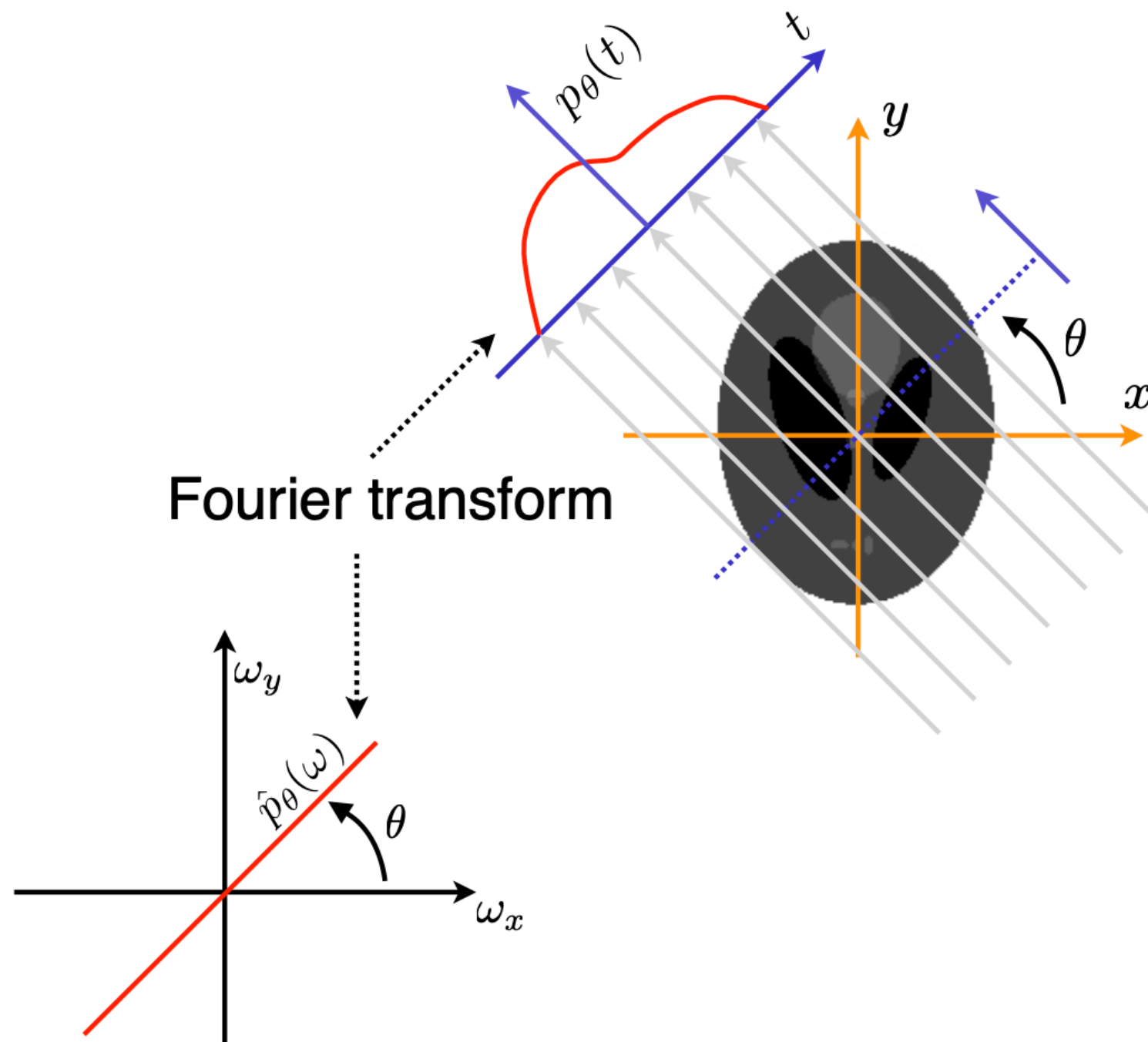
$$p_\theta(t) = \mathcal{R}\{f\}(\boldsymbol{\theta}, t) = \int_{\mathbb{R}^2} f(\boldsymbol{x}) \delta(\boldsymbol{\theta}^\top \boldsymbol{x} - t) \, d\boldsymbol{x}$$

How can we even compute this?

Theorem: Fourier slice theorem

$$\hat{p}_\theta(\omega) = \hat{f}(\omega \cos \theta, \omega \sin \theta)$$

Radon Transform



Steerability of Polynomials

Property: The rotated version of a 2D polynomial of degree p is a 2D polynomial of degree p . This implies that polynomials are “steerable”.

Why is this useful?

How do we establish this property?

Key observations for establishing this property:

- A 2D polynomial of degree p is a linear combination of monomials of degree $n \leq p$: $x^{k_1}y^{k_2}$ with $k_1 + k_2 = n$
- A rotation of a monomial of degree k yields a polynomial of degree k

A rotation of a polynomial is a polynomial of the same degree

Gradient and Directional Derivatives

Direction specified by $\mathbf{u} \in \mathbb{R}^2$ with $\|\mathbf{u}\|_2 = 1$ (unit vector)

- First-order directional derivatives

$$D_{\mathbf{u}}f(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x}) - f(\mathbf{x} - h\mathbf{u})}{h}$$

$$= \mathbf{u}^T \nabla f(\mathbf{x})$$

$$\xleftrightarrow{\mathcal{F}}$$

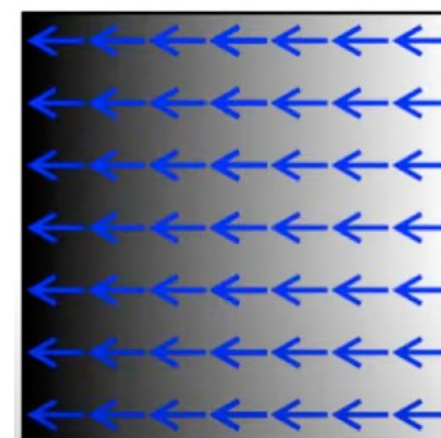
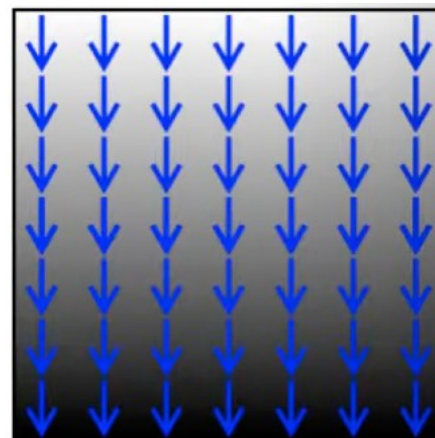
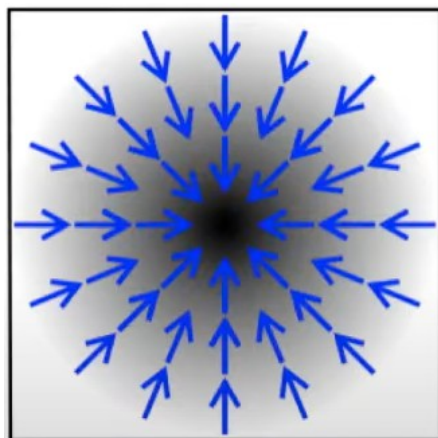
$$\mathbf{j} \mathbf{u}^T \boldsymbol{\omega} \hat{f}(\boldsymbol{\omega})$$

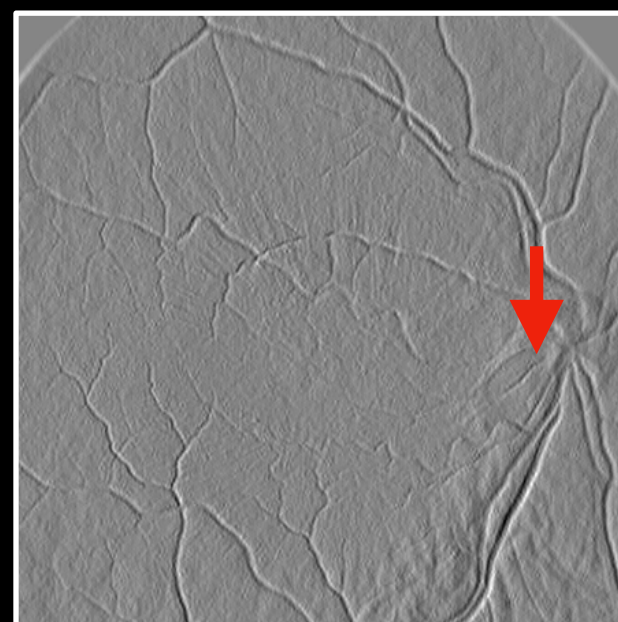
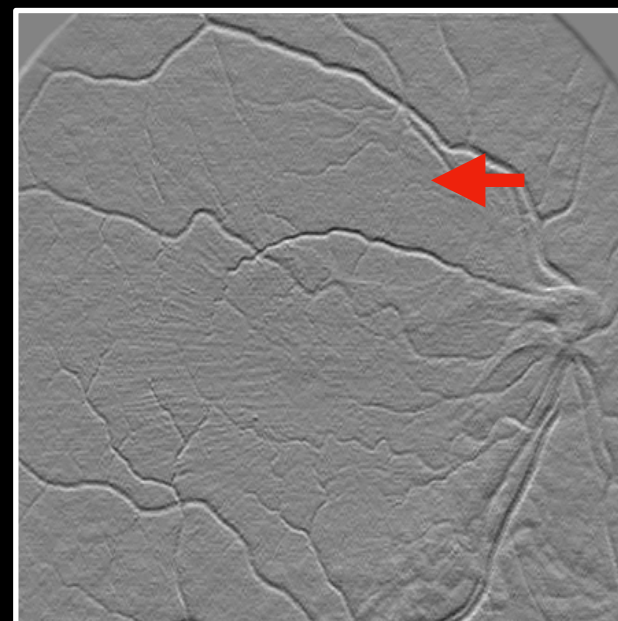
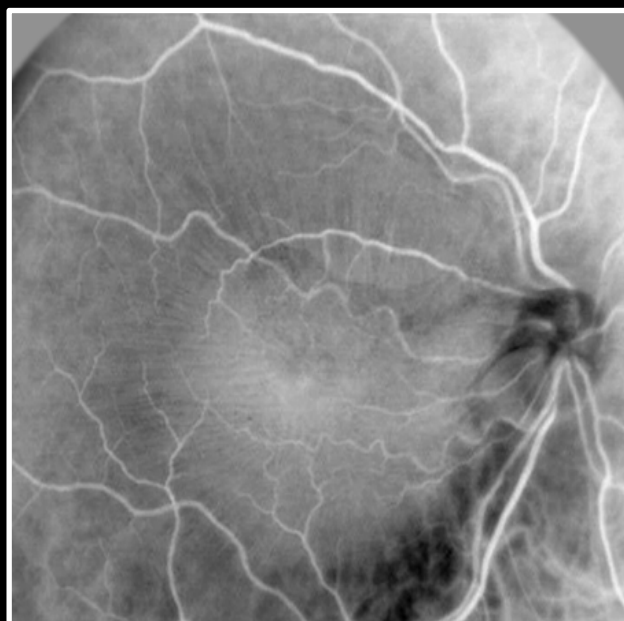
$$= u_1 \frac{\partial f(\mathbf{x})}{\partial x} + u_2 \frac{\partial f(\mathbf{x})}{\partial y}$$

$$\xleftrightarrow{\mathcal{F}}$$

$$\mathbf{j}(u_1 \omega_1 + u_2 \omega_2) \hat{f}(\boldsymbol{\omega})$$

$$\mathbf{n}(\mathbf{x}) = \frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|_2} \text{ maximizes the directional derivative}$$





Higher-Order Directional Derivatives

Direction specified by $\mathbf{u} \in \mathbb{R}^2$ with $\|\mathbf{u}\|_2 = 1$ (unit vector)

- Directional derivative of order n

$$D_{\mathbf{u}}^n f(\mathbf{x}) = \underbrace{D_{\mathbf{u}} D_{\mathbf{u}} \cdots D_{\mathbf{u}}}_{n \text{ times}} f(\mathbf{x}) \quad \xleftrightarrow{\mathcal{F}} \quad (\mathbf{j} \mathbf{u}^\top \boldsymbol{\omega})^n \hat{f}(\boldsymbol{\omega})$$

Exercise: Let $\mathbf{u}_\theta = (\cos \theta, \sin \theta)$. Explicitly determine $D_{\mathbf{u}_\theta}^2 f(\mathbf{x})$ as a function of θ and partial derivatives of f .

Directional Image Analysis

- Structure Tensor
- Implementation
- Examples of 2D Directional Analysis

Structure Tensor

- Structure tensor at location $\mathbf{x}_0$

$$\mathbf{J}(\mathbf{x}_0) = \int_{\mathbb{R}^2} w(\mathbf{x} - \mathbf{x}_0) \nabla f(\mathbf{x}) \nabla f(\mathbf{x})^\top d\mathbf{x}$$

- $w(\mathbf{x})$: nonnegative symmetric “observation window” (e.g., Gaussian)

- $\mathbf{J}$: 2×2 symmetric matrix

Why are the eigenvalues real?

- Eigenvectors and eigenvalues: $\mathbf{J}\mathbf{u}_i = \lambda_i \mathbf{u}_i$, $i = 1, 2$ with $\lambda_1 \geq \lambda_2$

- Interpretation for window centered at $\mathbf{x}_0 = \mathbf{0}$

- Weighted inner product

$$\mathbf{J} = \langle \nabla f, \nabla f \rangle_w$$

e.g., $[\mathbf{J}]_{1,1} = \left\langle \frac{\partial f}{\partial x}, \frac{\partial f}{\partial x} \right\rangle_w$ with $\langle f_1, f_2 \rangle_w = \int_{\mathbb{R}^2} w(\mathbf{x}) f_1(\mathbf{x}) f_2(\mathbf{x}) d\mathbf{x}$

- Energy of u -directional derivative

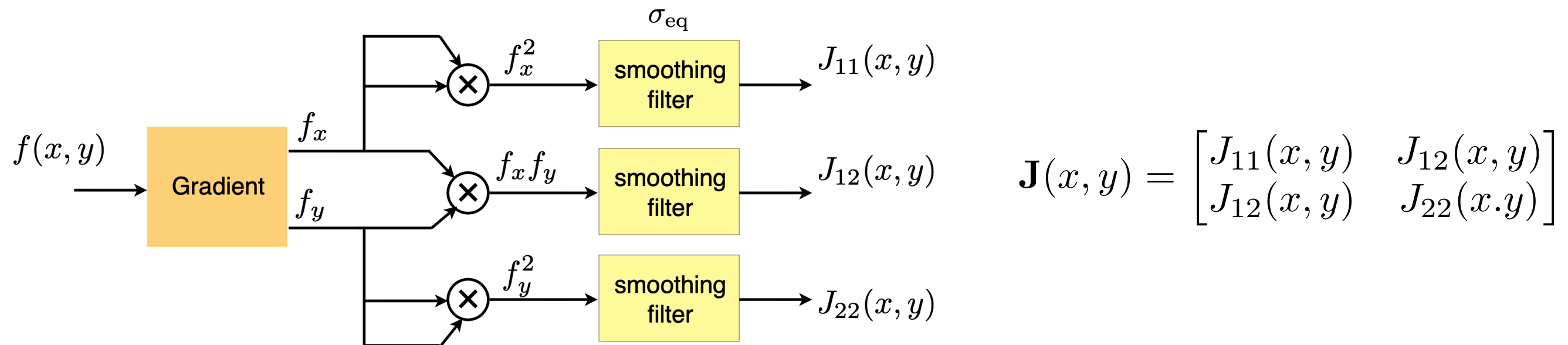
$$\|\mathbf{D}_{\mathbf{u}} f\|_w^2 = \langle \mathbf{u}^\top \nabla f, \mathbf{u}^\top \nabla f \rangle_w = \mathbf{u}^\top \langle \nabla f, \nabla f \rangle_w \mathbf{u} = \mathbf{u}^\top \mathbf{J} \mathbf{u}$$

- Dominant orientation of a neighborhood: $\mathbf{u}_1 = \operatorname{argmax}_{\|\mathbf{u}\|=1} \|\mathbf{D}_{\mathbf{u}} f\|_w^2$

$$\text{Eigenvalues: } \lambda_i = \mathbf{u}_i^\top \mathbf{J} \mathbf{u}_i$$

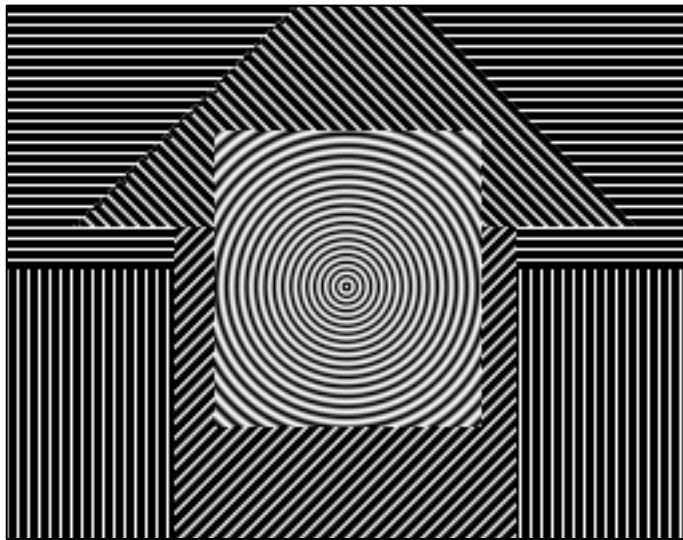
Structure Tensor Implementation

Exercise: How would you implement the structure tensor?

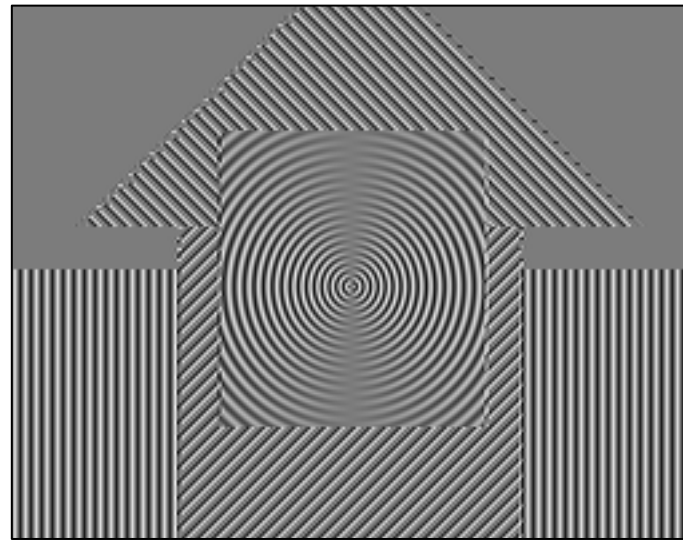


- Structure tensor allows us to understand **local features**
 - Gradient “energy”: $E = \text{trace}(\mathbf{J}) = J_{11} + J_{22}$
 - Orientation: $\mathbf{u}_1 = (\cos \theta, \sin \theta)$ with $\theta = \frac{1}{2} \arctan \left(\frac{2J_{12}}{J_{22} - J_{11}} \right)$
 - Coherency: $0 \leq C = \frac{\lambda_1 - \lambda_2}{\lambda_1 + \lambda_2} = \frac{\sqrt{(J_{22} - J_{11})^2 + 4J_{12}^2}}{J_{22} + J_{11}} \leq 1$
 - Harris corner index: $H = \det(\mathbf{J}) - \kappa \text{trace}(\mathbf{J})^2$ with $\kappa \in [0.04, 0.06]$

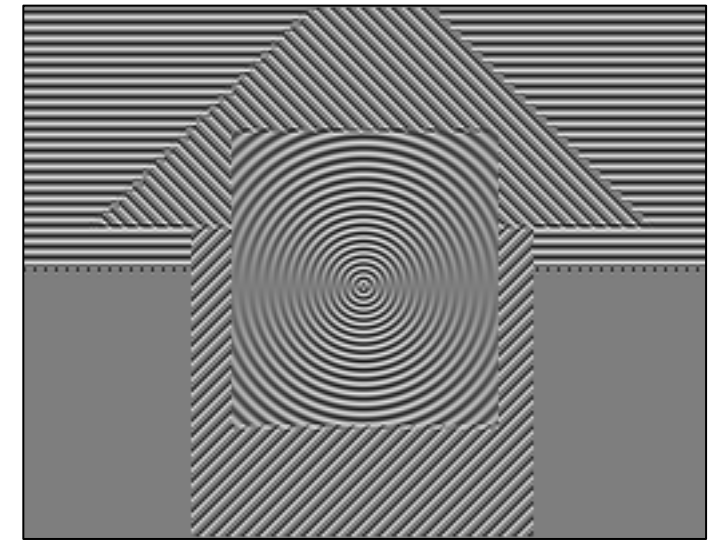
Examples of Directional Analysis



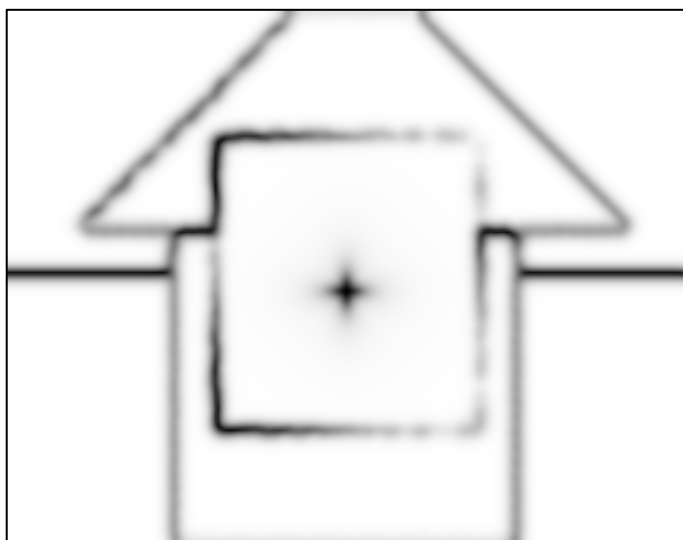
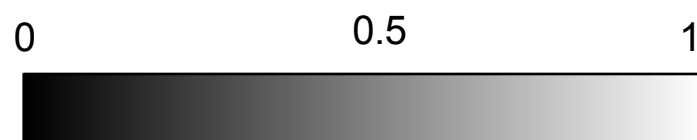
Input



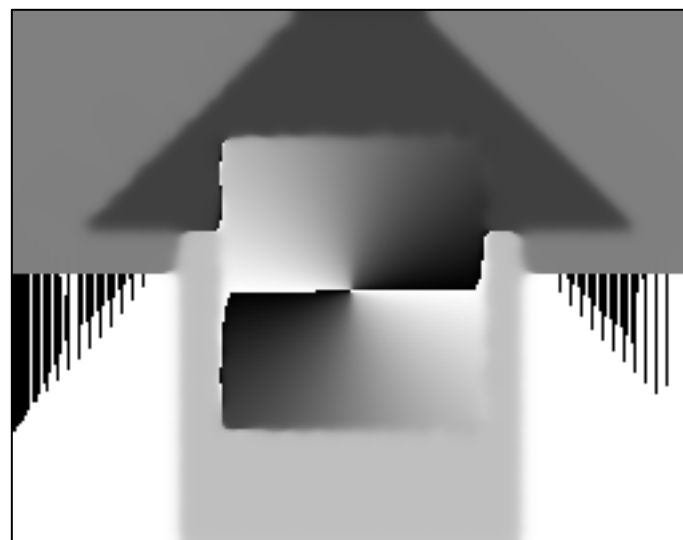
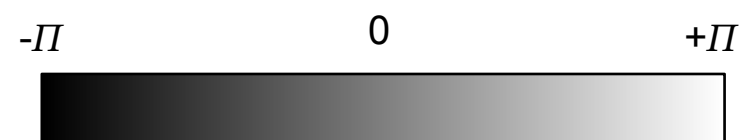
Dx



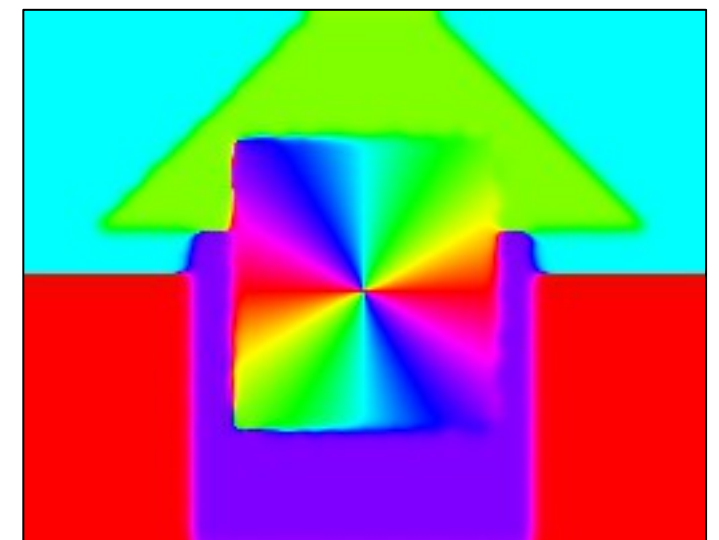
Dy



Coherency



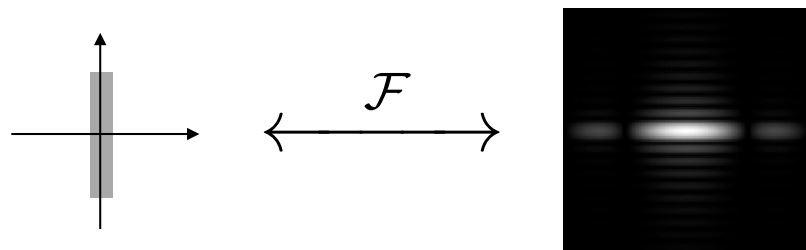
Orientation



Orientation

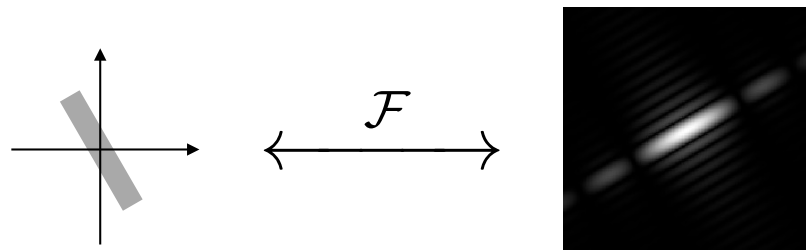
Orientation Estimation: Revisited

- **Problem:** Design a (real time?) system that can determine the orientation of a (linear) pattern placed at an arbitrary location in an image.



$$g_{\theta}(\mathbf{x}) = f(\mathbf{R}_{\theta}\mathbf{x})$$

$$\mathbf{R}_{\theta} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$



$$g_{\theta}(\mathbf{x}) \xleftrightarrow{\mathcal{F}} \hat{f}(\mathbf{R}_{\theta}\boldsymbol{\omega})$$

We want to find the orientation in the Fourier domain with the **least spread**.

Problem Solution

- Compute the “Fourier inertia” matrix (second-moment matrix)

$$\mathbf{M} = \begin{bmatrix} \iint \omega_1^2 |\hat{f}(\boldsymbol{\omega})|^2 d\omega_1 d\omega_2 & \iint \omega_1 \omega_2 |\hat{f}(\boldsymbol{\omega})|^2 d\omega_1 d\omega_2 \\ \iint \omega_2 \omega_1 |\hat{f}(\boldsymbol{\omega})|^2 d\omega_1 d\omega_2 & \iint \omega_2^2 |\hat{f}(\boldsymbol{\omega})|^2 d\omega_1 d\omega_2 \end{bmatrix}$$
$$= \begin{bmatrix} \langle j\omega_1 \hat{f}(\boldsymbol{\omega}), j\omega_1 \hat{f}(\boldsymbol{\omega}) \rangle & \langle j\omega_1 \hat{f}(\boldsymbol{\omega}), j\omega_2 \hat{f}(\boldsymbol{\omega}) \rangle \\ \langle j\omega_2 \hat{f}(\boldsymbol{\omega}), j\omega_1 \hat{f}(\boldsymbol{\omega}) \rangle & \langle j\omega_2 \hat{f}(\boldsymbol{\omega}), j\omega_2 \hat{f}(\boldsymbol{\omega}) \rangle \end{bmatrix}$$

Second-order moments measure spread

$$= (2\pi)^2 \begin{bmatrix} \langle \partial_x f, \partial_x f \rangle & \langle \partial_x f, \partial_y f \rangle \\ \langle \partial_y f, \partial_x f \rangle & \langle \partial_y f, \partial_y f \rangle \end{bmatrix} \quad (\text{fast algorithm via Parseval-Plancherel})$$

Which direction will have the least spread?

The direction of the **smallest eigenvalue**

Problem Solution (cont'd)

- Eigendecomposition of $\mathbf{M}$ gives us the **axes of inertia**

$$\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} = \begin{bmatrix} \mathbf{u}_1^T \\ \mathbf{u}_2^T \end{bmatrix} \mathbf{M} \begin{bmatrix} \mathbf{u}_1 & \mathbf{u}_2 \end{bmatrix}$$

$$\lambda_1 \geq \lambda_2$$

$\mathbf{u}_1$: eigenvector in the direction of the **long** axis

$\mathbf{u}_2$: eigenvector in the direction of the **short** axis

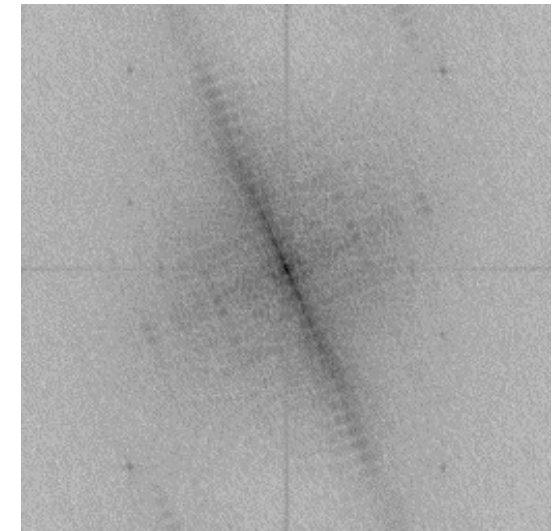
- Pipeline:
 1. Compute the Fourier inertia matrix $\mathbf{M}$ via the fast algorithm
 2. Compute the eigendecomposition of $\mathbf{M}$ and store $\mathbf{u}_2$
 3. Return the angle of $\mathbf{u}_2$
 - * $\theta = \arctan \frac{u_{22}}{u_{21}}$

Orientation Estimation in Action

- Image 1:

Measured angle: $25^\circ \pm 2^\circ$

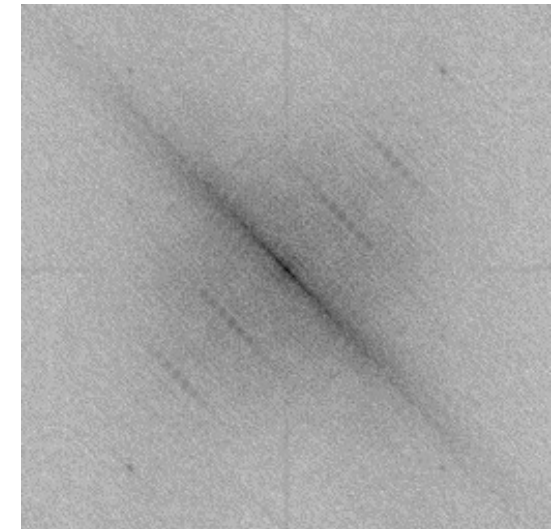
Computed angle: 27°

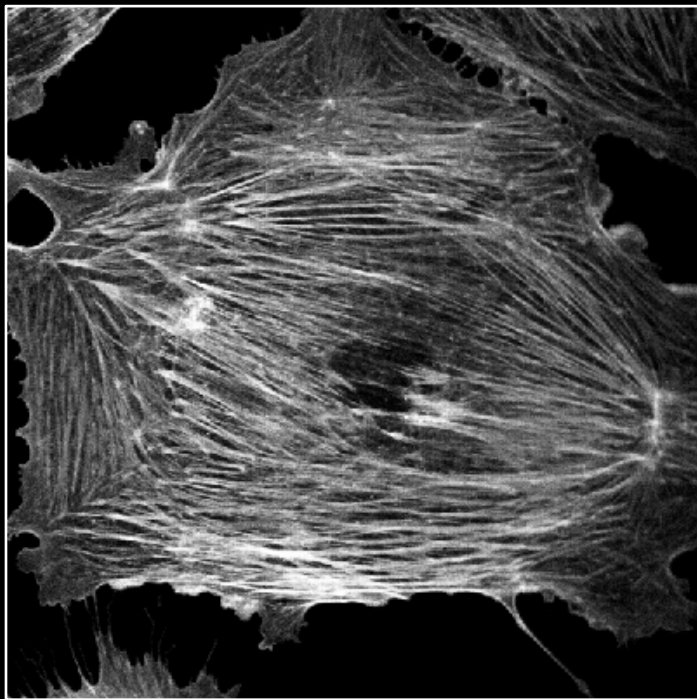


- Image 2:

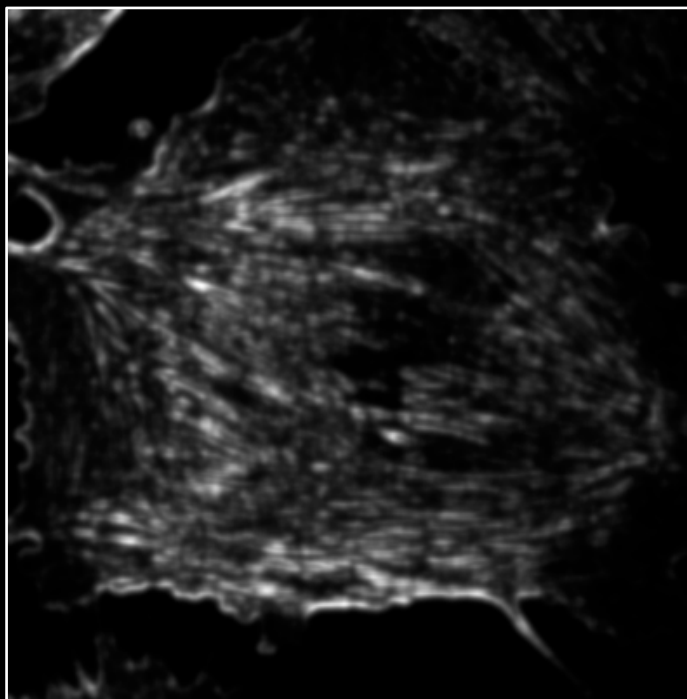
Measured angle: $44^\circ \pm 2^\circ$

Computed angle: 45.6°





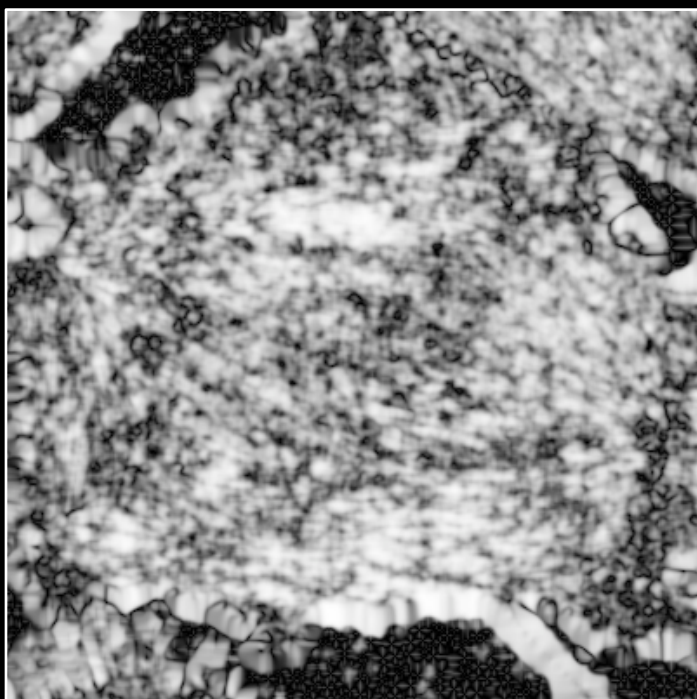
Input



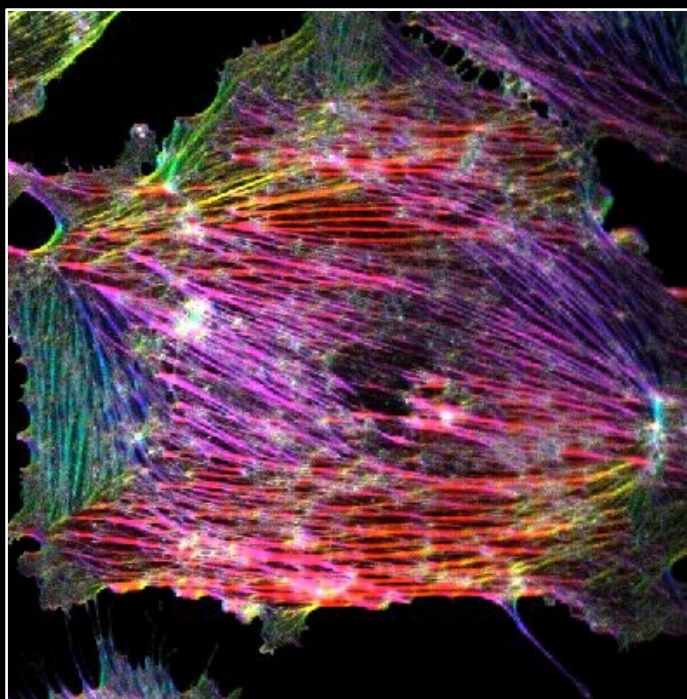
Energy



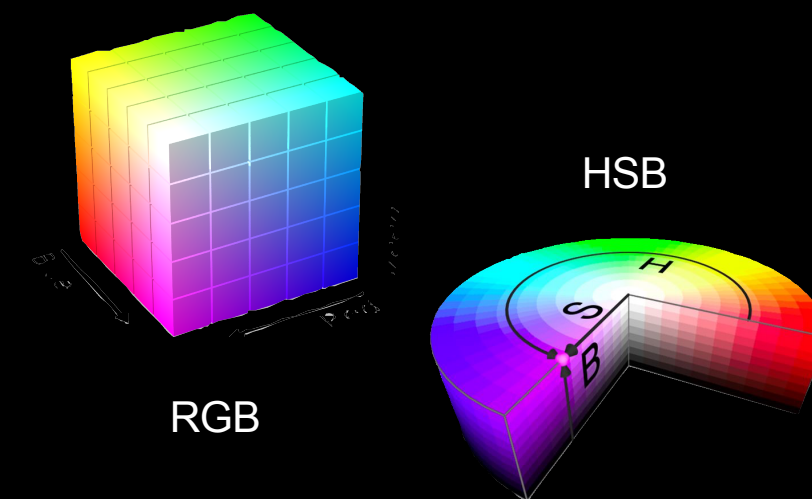
Orientation



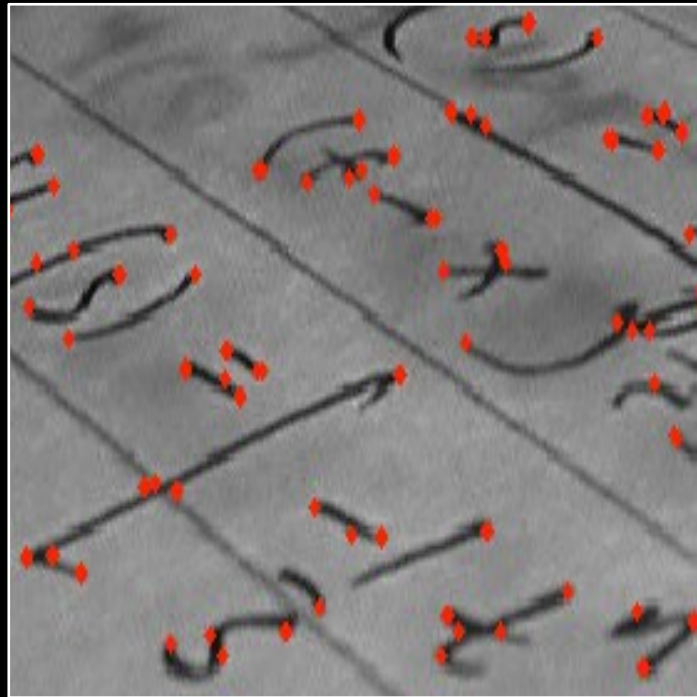
Coherency



Color HSB
representation



Hue: orientation
Saturation: coherency
Brightness: input



Keypoints detector
(Harris Corner)

Steerable Filters

- Directional Pattern Matching
- Steerable Filters
- Derivative Filters

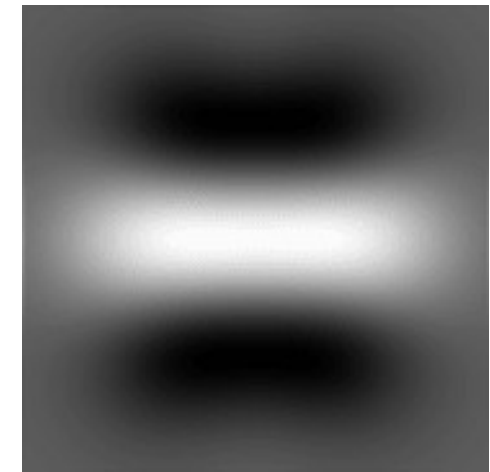
Directional Pattern Matching

Task: detection/enhancement of a given type of directional pattern

Example: edge, line, ridge, filament, corner, etc.

- Measurement model (signal + noise): $f(\mathbf{x}) = I f_0(\mathbf{R}_\theta(\mathbf{x} - \mathbf{x}_0)) + n(\mathbf{x})$

- $f_0(\mathbf{x})$: template (e.g., elongated blob)
- $\mathbf{x}_0$: spatial location (unknown)
- $\mathbf{R}_\theta$: 2×2 rotation by θ (unknown)
- I : intensity (unknown)
- $n(\mathbf{x})$: additive white Gaussian noise



Have we seen this problem before?

- Maximum-likelihood estimator (rotating matched filter)

Define $h(\mathbf{x}) = f_0(-\mathbf{x})$ and $h_\theta(\mathbf{x}) = h(\mathbf{R}_\theta \mathbf{x})$

$$\tilde{\theta}(\mathbf{x}) = \operatorname{argmax}_\theta (f * h_\theta)(\mathbf{x})$$

$$\tilde{I}(\mathbf{x}) = (f * h_{\tilde{\theta}(\mathbf{x})})(\mathbf{x})$$

Why is this approach bad?

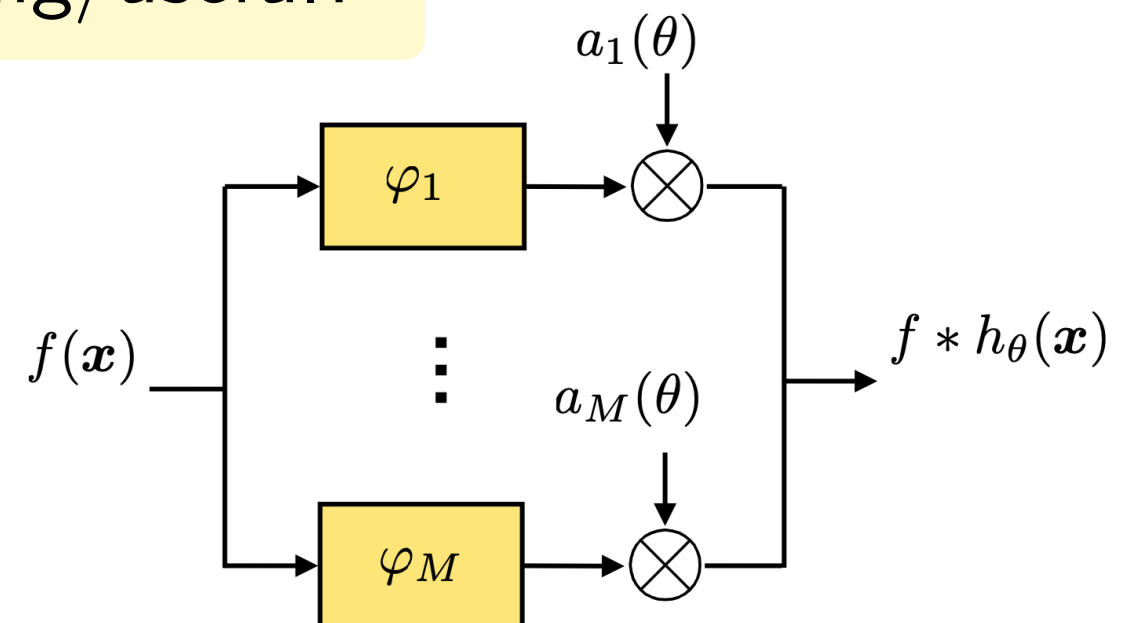
computationally expensive

Definition: A 2D filter $h(\mathbf{x})$, $\mathbf{x} \in \mathbb{R}^2$ is steerable of order M if and only if there exist “basis filters” $\varphi_m(\mathbf{x})$ and coefficients $a_m(\theta)$ such that

$$h_\theta(\mathbf{x}) = h(\mathbf{R}_\theta \mathbf{x}) = \sum_{m=1}^M a_m(\theta) \varphi_m(\mathbf{x}) \quad \text{for all } \theta \in [-\pi, \pi]$$

Why is this interesting/useful?

- Fast implementation



Exercise: Prove that $h(\mathbf{x})$ is steerable $\Leftrightarrow \hat{h}(\boldsymbol{\omega})$ is steerable

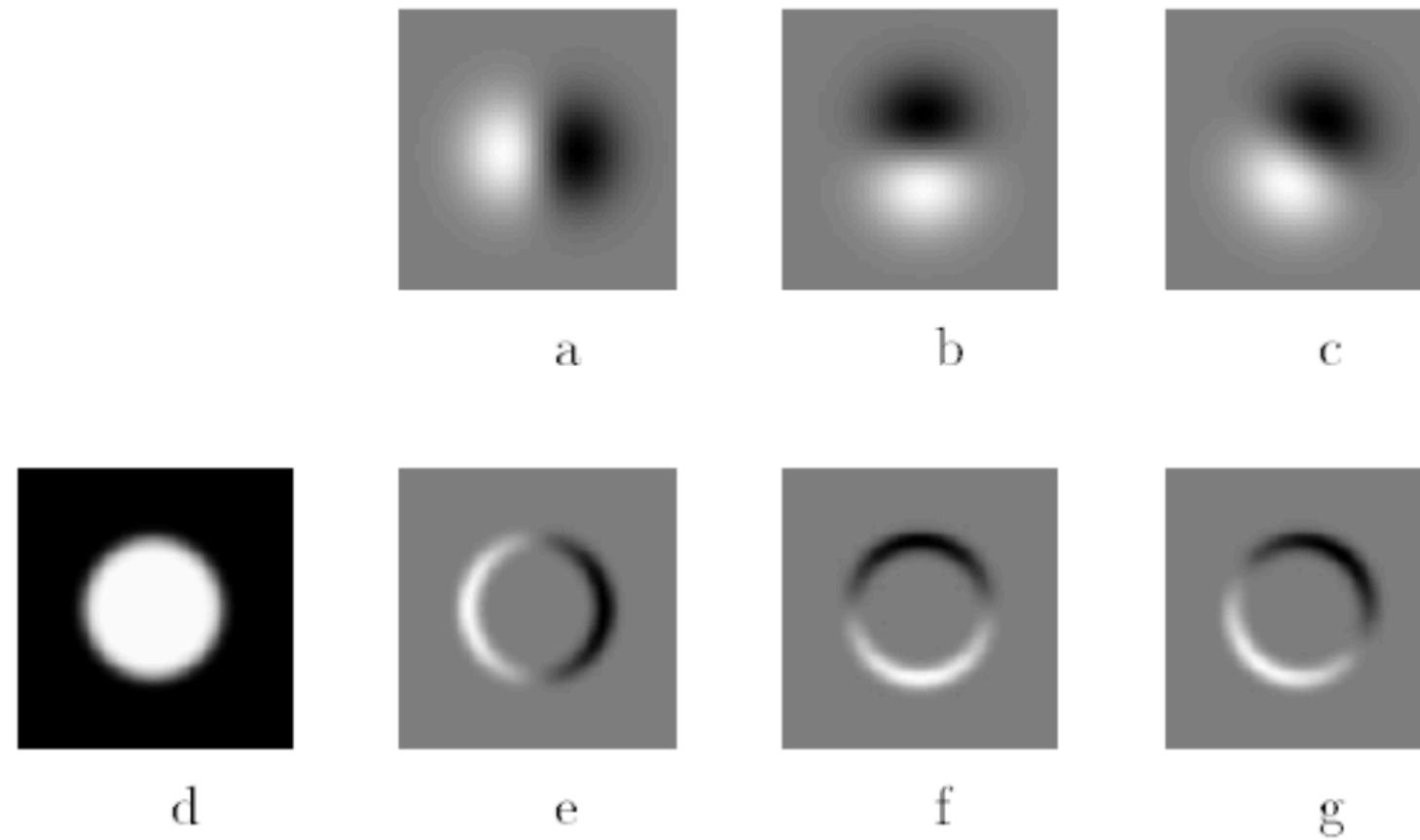
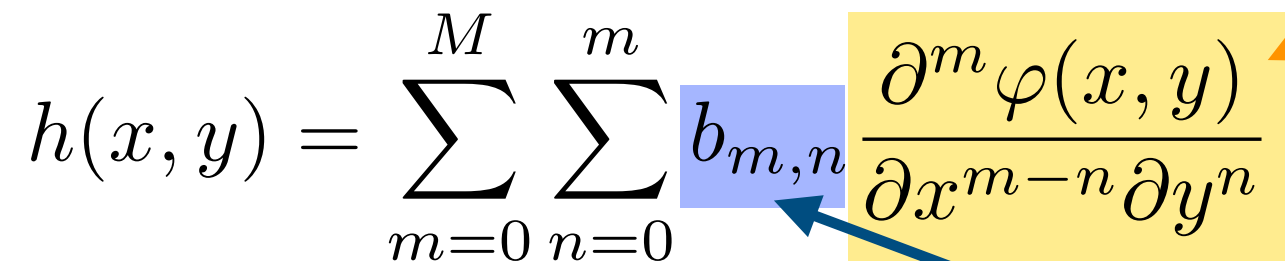


Fig. 1. Example of steerable filters: (a) $G_1^{0^\circ}$ first derivative with respect to x (horizontal) of a Gaussian; (b) $G_1^{90^\circ}$, which is $G_1^{0^\circ}$, rotated by 90° . From a linear combination of these two filters, one can create G_1^θ , which is an arbitrary rotation of the first derivative of a Gaussian; (c) $G_1^{60^\circ}$, formed by $\frac{1}{2}G_1^{0^\circ} + \frac{\sqrt{3}}{2}G_1^{90^\circ}$. The same linear combinations used to synthesize G_1^θ from the basis filters will also synthesize the response of an image to G_1^θ from the responses of the image to the basis filters; (d) image of circular disk; (e) $G_1^{0^\circ}$ (at a smaller scale than pictured above) convolved with the disk (d); (f) $G_1^{90^\circ}$ convolved with (d); (g) $G_1^{60^\circ}$ convolved with (d), obtained from $\frac{1}{2}$ (image (e)) + $\frac{\sqrt{3}}{2}$ (image (f)).

Steerable Filter Design

Isotropic low-pass function (e.g., Gaussian): $\varphi(x, y)$

Subspace of steerable derivative-based templates:

$$h(x, y) = \sum_{m=0}^M \sum_{n=0}^m b_{m,n} \frac{\partial^m \varphi(x, y)}{\partial x^{m-n} \partial y^n}$$


basis functions

expansion coefficients

Exercise: Prove that this is steerable.

$$\varphi(\mathbf{x}) \text{ isotropic} \Leftrightarrow \varphi(\mathbf{x}) = \varphi_{\text{iso}}(\|\mathbf{x}\|_2) \Leftrightarrow \hat{\varphi}(\boldsymbol{\omega}) = \rho(\|\boldsymbol{\omega}\|_2)$$

$$h(\mathbf{x}) \text{ steerable} \Leftrightarrow \hat{h}(\boldsymbol{\omega}) \text{ steerable}$$

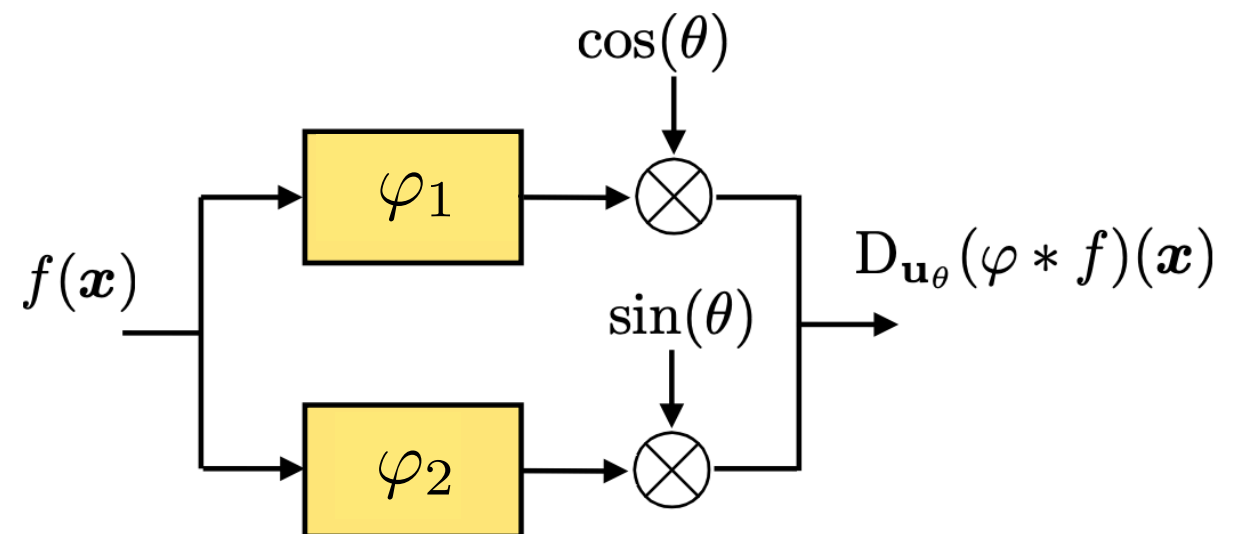
Steerable Filters for Edge and Ridge Detection

<https://bigwww.epfl.ch/demo/ip/demos/edgeDetector/>

- Gradient-based edge detector

$$h(\mathbf{x}) = \varphi_1(\mathbf{x}) = \frac{\partial \varphi(\mathbf{x})}{\partial x}$$

$$\varphi_2(\mathbf{x}) = \frac{\partial \varphi(\mathbf{x})}{\partial y}$$

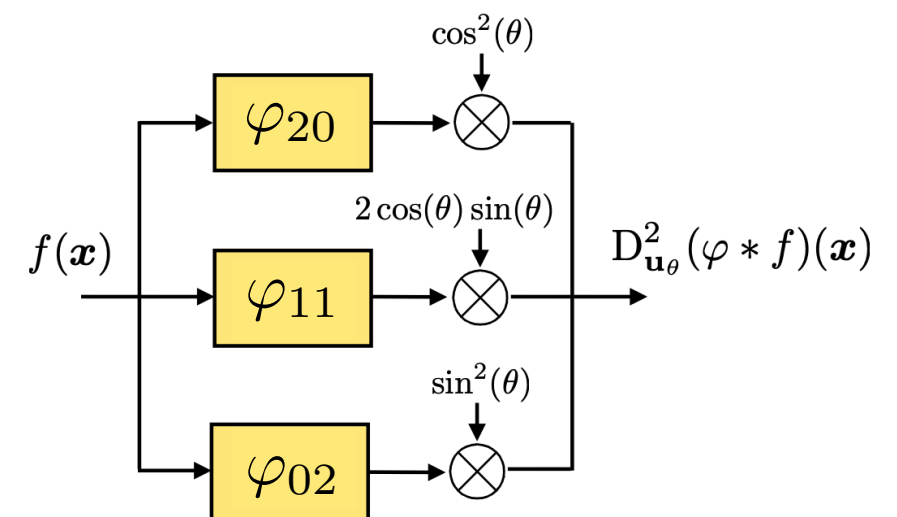


$$h(\mathbf{R}_\theta \mathbf{x}) = D_{\mathbf{u}_\theta} \varphi(\mathbf{x}) = \mathbf{u}_\theta^\top \nabla \varphi(\mathbf{x}) = \cos \theta \varphi_1(\mathbf{x}) + \sin \theta \varphi_2(\mathbf{x})$$

- Second-order derivatives = ridge detector

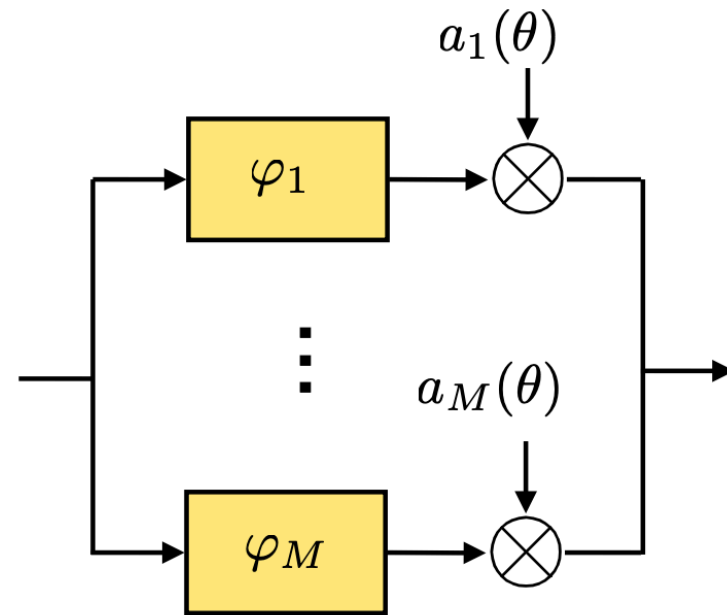
$$h(\mathbf{x}) = \varphi_{20}(\mathbf{x}) = \frac{\partial^2 \varphi(\mathbf{x})}{\partial x^2}$$

$\varphi_{02}, \varphi_{11}$

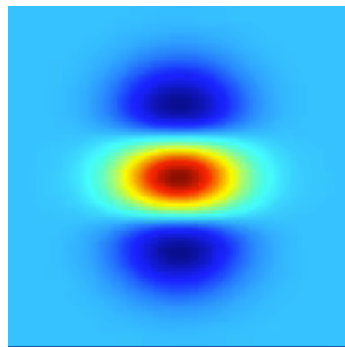
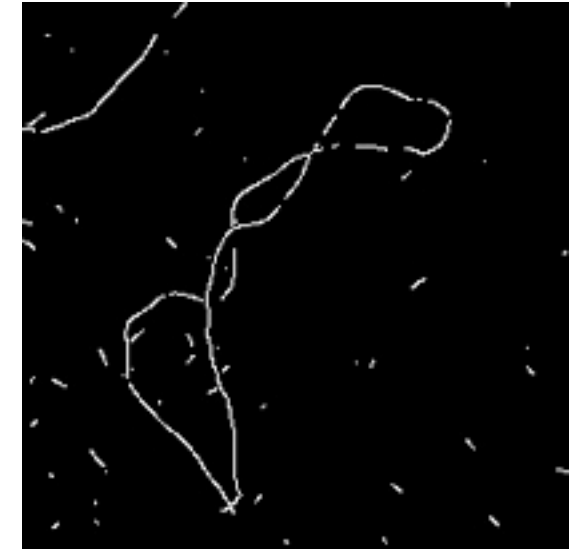


$$h(\mathbf{R}_\theta \mathbf{x}) = D_{\mathbf{u}_\theta}^2 \varphi(\mathbf{x}) = (\cos \theta)^2 \varphi_{20}(\mathbf{x}) + 2 \cos \theta \sin \theta \varphi_{11}(\mathbf{x}) + (\sin \theta)^2 \varphi_{02}(\mathbf{x})$$

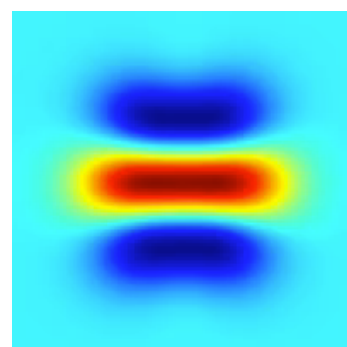
Ridge Detection Example



$$\theta^*(\mathbf{x}) = \arg \max_{\theta} \{ (h_{\theta} * f)(\mathbf{x}) \}$$



2nd order



4th order

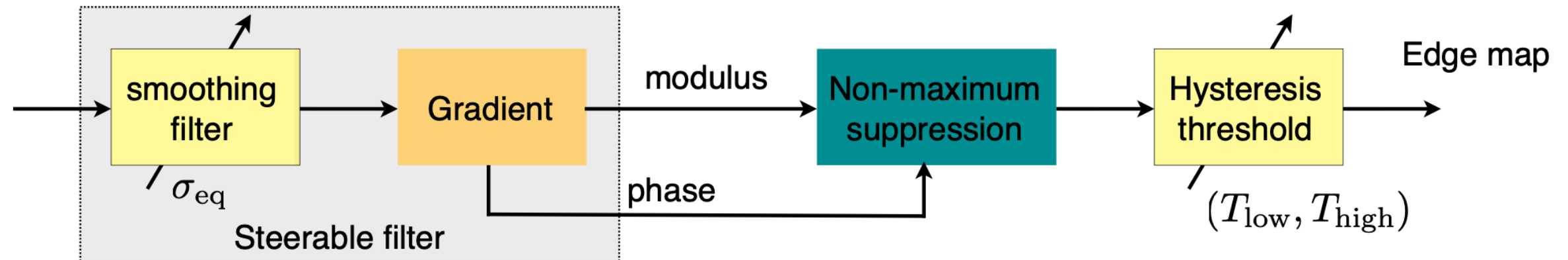


without steering

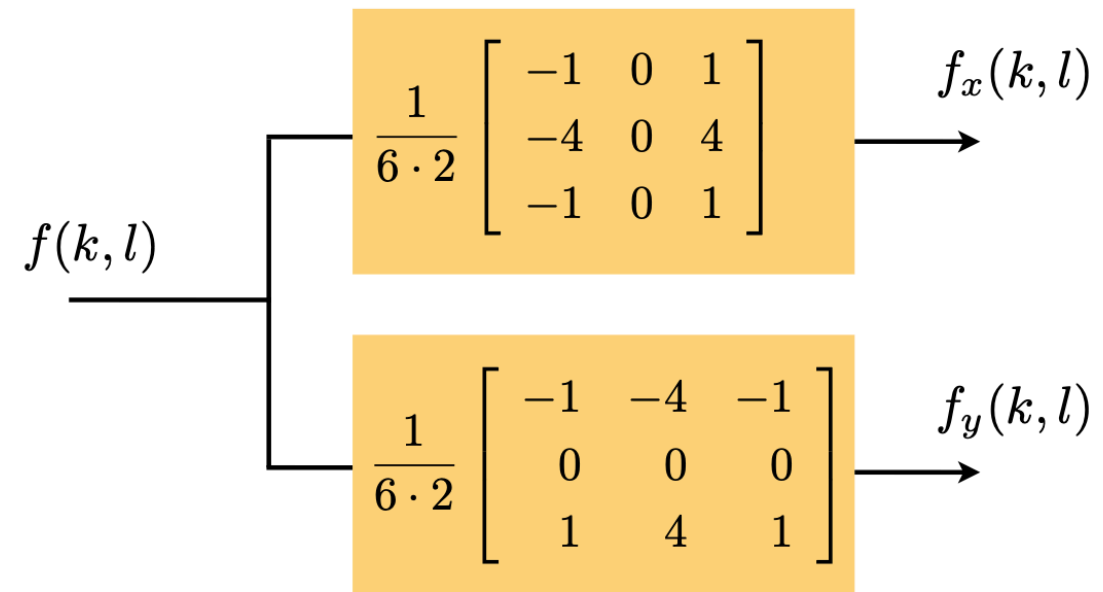
Canny Edge Detector Revisited

- State-of-the-art edge detector

Edge point = local maximum of first directional derivative



- Smoothing
 - Gaussian filter: isotropic + separable (the only one)
 - Implementation: cascade of simple recursive filters
- Discrete gradient filters



Canny Edge Detector Revisited

- Non-maximum suppression at $\mathbf{x}_0$

$$\mathbf{u} = \frac{\nabla f(\mathbf{x}_0)}{\|\nabla f(\mathbf{x}_0)\|_2} : \text{unit vector in gradient direction}$$

$$\begin{aligned} \text{if } \|\nabla f(\mathbf{x}_0)\|_2 \geq \|\nabla f(\mathbf{x}_0 \pm \mathbf{u})\|_2 \quad & \text{then } g(\mathbf{x}_0) = \|\nabla f(\mathbf{x}_0)\|_2 \\ & \text{else } g(\mathbf{x}_0) = 0 \end{aligned}$$

- Hysteresis threshold

Set of points: $\mathbf{k} \in \mathbb{Z}^2$

Two auxiliary edge maps:

$$\text{Gray square} \quad E_{\text{low}} = \{\mathbf{k} : T_{\text{low}} \leq g[\mathbf{k}] \leq T_{\text{high}}\}$$

$$\text{Blue square} \quad E_{\text{high}} = \{\mathbf{k} : T_{\text{high}} \leq g[\mathbf{k}]\}$$

Final edge map:

$$\text{Orange square} \quad E = \{\mathbf{k} \in E_{\text{low}} \cup E_{\text{high}} : \text{there exists a path that connects } \mathbf{k} \text{ to } E_{\text{high}}\}$$

